CSA02 – ASSIGNMENT 1

Grama Jyothi Rural Electricity Co-operative

Individual Assignment

Name: BAVADARANI S
Register Number:1912260562
Course / Section: CSA0209 /C PROGRAMMING-SLOT B
GitHub Repository Link: https://github.com/bavadarini28/ CSA0209-1912260562-BAVADARANI-S-ASSIGNMENT-1.git

TASK 1 – Understand the Problem and Identify the Data (4 Marks)

1.1 Problem Analysis

Grama Jyothi needs a reliable electricity billing system that calculates monthly consumption and bills for households. It must avoid incorrect bills caused by floating-point precision errors, meter rollover, and inconsistent tariff values. The program calculates consumption from previous and current readings, applies the energy tariff slabs, and adds a fixed charge. The final bill must be accurate to the nearest paisa.

1.2 Input Data

Data item	Purpose	Suitable representation
Household number	Uniquely identifies each house.	unsigned int, or string if it contains letters
Previous reading	Meter reading at the start of the billing period.	meter_t (unsigned short)
Current reading	Latest meter reading.	meter_t (unsigned short)
Household category	Selects the applicable tariff category.	enum or integer code
Tariff rates	Determine the charge per unit.	Integer paise
Fixed charge	Amount charged regardless of consumption.	Integer paise

1.3 Output Data

Output	Purpose
Units consumed	Difference between readings, accounting for rollover.
Energy charge	Charge calculated using the applicable tariff slabs.
Fixed charge	Fixed amount added to the energy charge.
Total bill	Energy charge plus fixed charge, displayed in rupees and paise.

1.4 Constants and Variables

Constants include the meter maximum value (65535), slab limits (100 and 200 units), energy rates (0, 225 and 450 paise), and fixed charge (5000 paise). They should be defined centrally so that a tariff revision can be made in one place. Variables include household number, readings, consumption, energy charge, and total bill because their values vary by household. The number of households can also change as the system expands.

1.5 Negative, Fractional and Large Values

- Negative values: Valid consumption and bill amounts should not be negative. A lower current reading may indicate rollover or invalid data.
- Fractional values: Tariffs such as ₹2.25 should be represented as 225 paise to avoid floating-point rounding errors.
- Large values: The district may contain 800,000 households. Use a sufficiently wide integer type such as long long for accumulated totals, after checking the maximum possible value.

1.6 Rules for a Correct Bill

1. Consumption must account for meter rollover.
2. The first 100 units are free under the given tariff.
3. Units 101–200 cost ₹2.25 per unit.

4. Units above 200 cost ₹4.50 per unit.
5. The fixed charge is ₹50 per household.
6. The bill must never be negative.
7. Total bill = energy charge + fixed charge.
8. Monetary calculations must preserve exact paise values.
9. Invalid readings and invalid household records must be detected.

1.7 Assumptions

- Each meter is a 16-bit counter with a maximum reading of 65535.
- At most one rollover occurs between consecutive readings.
- Readings represent cumulative energy consumption.
- The stated tariff applies to every household unless its category specifies otherwise.
- The fixed charge is ₹50 per billing period.
- The program displays money to two decimal places.

1.8 Alternative Concept: Store Readings as Strings

Readings such as "01530" could be stored as character arrays. This preserves leading zeros, but the text should be validated and converted before arithmetic. Parsing or comparing a string of d digits takes O(d) time. A null-terminated string needs approximately d+1 bytes, while the specified 16-bit integer meter type uses 2 bytes. Numeric integer comparison and subtraction are O(1) for fixed-width values. Lexicographic string comparison can be incorrect unless strings have a consistent format and length.

TASK 2 – Choose the Data Types (4 Marks)

2.1 Data Type Selection

Data item / type	C type	Typical size*	Range / purpose
Meter reading	unsigned short	2 bytes	0 to 65,535 if 16-bit
Household number	unsigned int	4 bytes	0 to 4,294,967,295 if 32-bit
Consumption units	unsigned int	4 bytes	0 to 4,294,967,295 if 32-bit
Tariff / fixed charge	long	8 bytes	Integer paise
Energy charge	long	8 bytes	Calculated charge in paise
Accumulated total	long long	8 bytes	Large bill totals
Household category	enum	Usually 4 bytes	Named category code
char	char	1 byte	Character or digit
float	float	4 bytes	Approximate decimal values
double	double	8 bytes	Higher-precision approximate decimal values

*Typical sizes for GCC on a 64-bit Linux system; verify on your own computer using sizeof. Use <limits.h> for integer limits (INT_MAX, LONG_MAX, LLONG_MAX, USHRT_MAX) and <float.h> for floating-point limits (FLT_MAX, DBL_MAX, FLT_DIG, DBL_DIG).

2.2 Why Money Should Be Stored in Paise

Floating-point types cannot represent every decimal fraction exactly. Repeated calculations can introduce small rounding errors. Store ₹162.50 as 16250 paise and perform the tariff calculations using integers. For district-wide accumulated totals, use a sufficiently wide integer type and verify that it cannot overflow.

2.3 Complete the Code – Meter Rollover

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   /* Convert the result back to the 16-bit unsigned range. */
   return (unsigned short)(current - previous);
}

int main(void)
{
   printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
          sizeof(meter_t), (unsigned)USHRT_MAX);
   printf("Consumption: %u units\n", consumption(1200, 1530));
   printf("Consumption: %u units\n", consumption(65400, 120));
   return 0;
}
```

Expected output:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```



Explanation: The meter uses modulo-65536 arithmetic. For a change from 65400 to 120, consumption is (65536 − 65400) + 120 = 136 + 120 = 256 units. The cast converts the result to the 16-bit unsigned range. This assumes unsigned short is 16 bits and no more than one rollover occurred.

Compile and run:

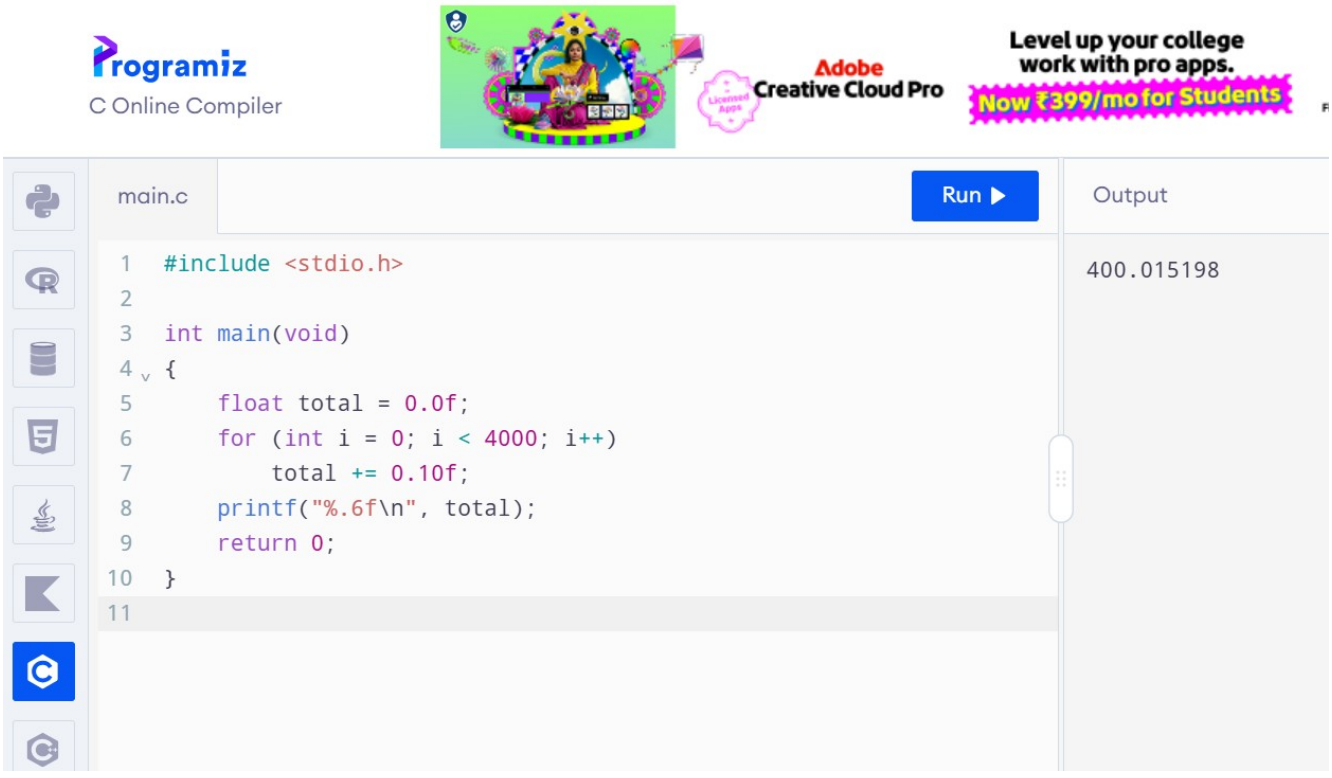
```
gcc -std=c11 -Wall -Wextra -o task02 task02.c
./task02
```

2.4 Float Accumulation Example

```
#include <stdio.h>

int main(void)
{
    float total = 0.0f;
    for (int i = 0; i < 4000; i++)
        total += 0.10f;
    printf("%.6f\n", total);
    return 0;
}
```

The printed result may differ slightly from 400.000000 because 0.10 is not exactly representable in binary floating point. In contrast, storing 10 paise per entry as an integer produces exactly 40,000 paise (₹400.00).



2.5 Alternative Concept: Double Rupees

Factor	Integer paise	double rupees
Addition time	O(1)	O(1)
Typical storage	8 bytes for long on assumed platform	8 bytes
Exact paise arithmetic	Yes, within integer range	Not guaranteed
Accumulated error	No fractional representation error	Small errors may accumulate
Display	Divide by 100	Round and format to 2 decimals

With double rupees, rounding should be applied consistently at the defined billing boundary, usually when the final bill is produced. Integer paise is preferred for predictable exact monetary arithmetic.

TASK 3 – Define the Constants (3 Marks)

3.1 Purpose of Constants

#define creates a preprocessor replacement and is useful for simple numeric values. const declares a typed object that cannot be modified through that identifier. enum defines named integer constants and is useful for household categories or status codes. Use uppercase names with underscores, such as RATE2_PAISE and FIXED_CHARGE_PAISE.

3.2 Complete the Code – Slab Billing

```
#include <stdio.h>


#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

/* Returns the bill in paise. */
long compute_bill(int units)
{
   long paise = FIXED_CHARGE_PAISE;

   if (units > SLAB2_UNITS) {
      paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
      paise += (long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
   } else if (units > SLAB1_UNITS) {
      paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
   }

   /* The first slab costs RATE1_PAISE, which is zero. */
   return paise;
}

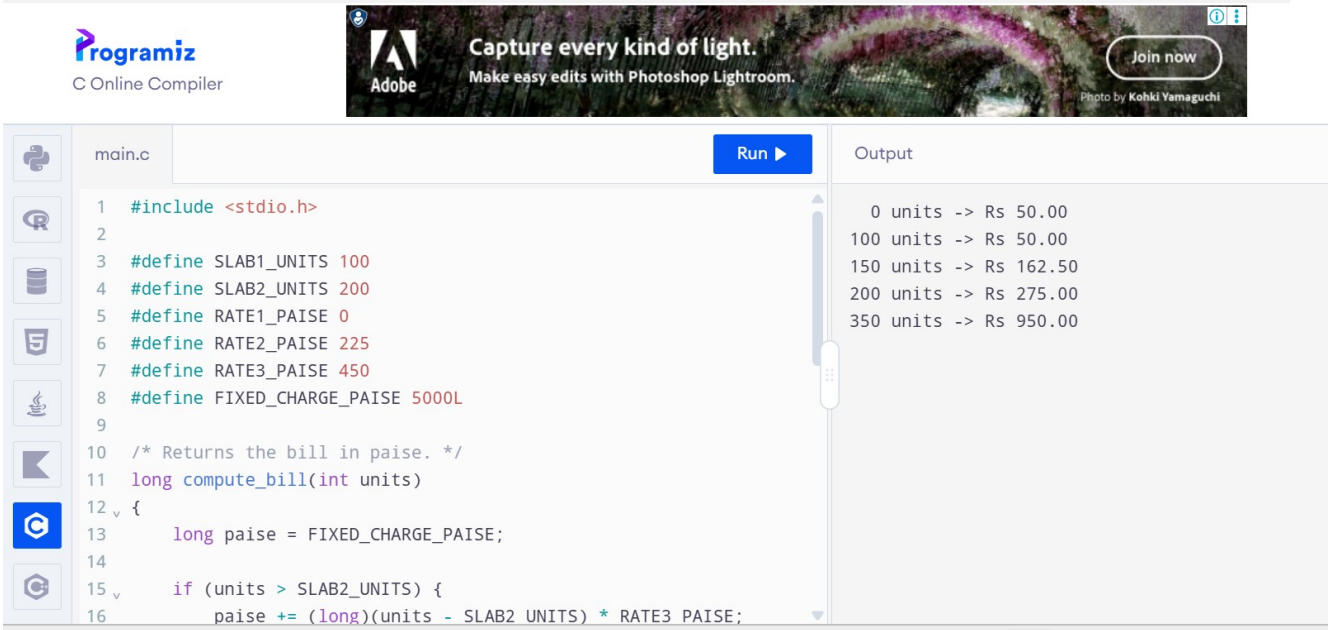
int main(void)
{
   int tests[] = {0, 100, 150, 200, 350};
   int n = sizeof tests / sizeof tests[0];
```

```
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }

    return 0;
}
```

Expected output:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```



Compile and run:

```
gcc -std=c11 -Wall -Wextra -o task03 task03.c
./task03
```

3.3 Tariff Revision Test

Change #define RATE2_PAISE 225 to #define RATE2_PAISE 230.

Units	Original bill	Revised bill
0	₹50.00	₹50.00
100	₹50.00	₹50.00

150	₹162.50	₹165.00
200	₹275.00	₹280.00
350	₹950.00	₹960.00

Only consumption in the second slab changes its rate. The fixed charge and third-slab rate remain unchanged.

3.4 Alternative Concept: const or enum

Typed const variables offer type checking and can be easier to inspect in a debugger. A const object may occupy storage, although optimisation can remove it. enum is suitable for named integer values, not fractional amounts. A #define normally does not create a variable or reserve storage for the constant itself. In typical optimised code, the choice need not affect running time. Macros are suitable for this assignment's simple tariff values.

TASK 4 – Use Macros Safely (4 Marks)

4.1 Definition of Macros

A macro is defined with #define. Before compilation, the preprocessor replaces macro names with their defined expressions. Macros can be used for repeated calculations such as converting rupees to paise, squaring a value, and finding the maximum of two values.

4.2 C Program Using Safe Macros

```
#include <stdio.h>

#define RUPEES_TO_PAISE(rupees) ((rupees) * 100L)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main(void)
{
int a = 4;
int b = 7;

printf("Rupees to paise: %ld\n", RUPEES_TO_PAISE(50));
printf("Square: %d\n", SQUARE(a + 1));
printf("Maximum: %d\n", MAX(a, b));

return 0;
}
```

Expected output:

```
Rupees to paise: 5000
Square: 25
Maximum: 7
```

Programiz
C Online Compiler



Get Premiere in
Creative Cloud Pro
Now ₹399/mo for Students
First year only. Annual plan, billed monthly. New subscribers only.
Learn more

main.c

Run ▶

```c

#define RUPEES_TO_PAISE(rupees) ((rupees) * 100L)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main(void)
{
    int a = 4;
    int b = 7;

    printf("Rupees to paise: %ld\n", RUPEES_TO_PAISE(50));
    printf("Square: %d\n", SQUARE(a + 1));
    printf("Maximum: %d\n", MAX(a, b));

    return 0;
}
```

Output

```
Rupees to paise: 5000
Square: 25
Maximum: 7
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra -o task04 task04.c
./task04
```

4.3 Why Parentheses Matter

Unsafe macro: #define SQUARE(x) x*x

SQUARE(a+1) expands to a+1*a+1. If a is 4, multiplication takes precedence and the result is 9, not 25. The safe definition #define SQUARE(x) ((x) * (x)) correctly groups the argument and the whole expression.

4.4 The Problem with MAX(a, b)

```
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

Calling MAX(i++, j) can evaluate i++ twice if that argument is selected: once in the comparison and once in the selected expression. This may change i unexpectedly. Avoid side-effect expressions in macro arguments; a function or inline function is safer.

4.5 Evidence Using gcc -E

```
gcc -E task04.c -o task04_preprocessed.i
gcc -E -P task04.c
```

The -E option runs only the preprocessor. The -P option omits line markers, making the expanded source easier to inspect. For example, SQUARE(a + 1) expands to ((a + 1) * (a + 1)). Save a screenshot of the terminal output.

4.6 When to Use a Function

Use a function for type checking, easier debugging, predictable argument evaluation, and more complex calculations. Macros are useful for simple textual substitutions, but functions are often safer for reusable calculations.

4.7 Alternative Concept: Static Inline Functions

```
#include <stdio.h>

static inline int square(int x)
{
    return x * x;
}

static inline int max_value(int a, int b)
{
    return (a > b) ? a : b;
}

int main(void)
{
    int a = 4;
    int b = 7;

    printf("Square: %d\n", square(a + 1));
    printf("Maximum: %d\n", max_value(a, b));

    return 0;
}
```

Expected output:

```
Square: 25
Maximum: 7
```

Programiz
C Online Compiler

Adobe
Creative Cloud Pro
Now ₹399/mo
for Students
Includes 20+ licensed apps.

main.c

Run ▶

Output

```c
#include <stdio.h>

static inline int square(int x)
{
    return x * x;
}

static inline int max_value(int a, int b)
{
    return (a > b) ? a : b;
}

int main(void)
{
    int a = 4;
    int b = 7;
```

```
Square: 25
Maximum: 7
```

4.8 Complexity Comparison

Factor	Macro	Static inline function
Execution time	Usually O(1) for these calculations	Usually O(1); may be inlined
Call overhead	No function call from expansion	May be avoided by inlining
Argument evaluation	Can happen more than once	Each argument is evaluated once before the call
Type checking	Limited to expanded expression	Normal C type checking
Code size	Expansion may be repeated at each use	Compiler may reuse or inline the implementation
Debugging	Can be difficult	Generally easier

If a macro is used k times, its expanded expression may appear k times in generated code. Actual machine-code size depends on compiler optimisation. A static inline function may reduce repetition, but the compiler is not required to inline it.

Final Submission Checklist

- Task 1: Problem analysis and alternative string representation.
- Task 2: Data types, rollover program and output screenshot.
- Task 3: Tariff macros, slab billing and revised-rate test.
- Task 4: Safe macros, inline functions and gcc -E evidence.
- README.md: Name, register number, GCC version, compile commands, run steps and results.
- GitHub repository named CSA02-<RegisterNumber>-Assignment1.
- At least one meaningful commit for each task.
- Working GitHub link on the report cover page and submission form.

Suggested GitHub Folder Structure

```
CSA02-RegisterNumber-Assignment1/
├── README.md
├── Task01/
│   └── analysis.md
├── Task02/
│   ├── task02.c
│   └── output.png
├── Task03/
│   ├── task03.c
│   ├── revised_output.png
│   └── output.png
└── Task04/
    ├── task04.c
    ├── task04_preprocessed.i
    └── output.png
```